Joint Activation and Structured Sparsity for Visual Wake Words on Akida 1

Akida 1 Model Zoo

August 2026

Abstract

Akida 1 hardware accelerates inference by exploiting *activation* (event) sparsity, not weight sparsity. A prior experiment (`SPARSITY_EXPERIMENT.md`) showed that VWW's ReLU activations can be pushed to roughly 70% sparsity for a modest accuracy cost using an activity regularizer. This report investigates a second, independent lever — *structured* (channel/filter) sparsity, which reduces the model's parameter count rather than its runtime behavior — and asks whether the two axes can be pushed together without the combined accuracy cost exceeding what either costs alone. We find that they are complementary rather than additive: a joint configuration reaches 77.6% activation sparsity and a 50% parameter reduction for a 3.90-point accuracy drop, less than the 4.86-point drop a naive sum of the two individual costs would predict. A lighter joint configuration further reduces this to a 1.22-point drop while retaining most of the sparsity and size benefit, making it the recommended operating point.

Contents

1 Introduction **1**

2 Background **2**
2.1 Akida 1 and activation sparsity . . . 2
2.2 Visual Wake Words . . . 2

3 Dataset **2**

4 Model architecture **3**

5 Methodology **4**
5.1 Activation sparsity: normalized Hoyer-Square regularization . . . 4
5.2 Structured sparsity: Network Slimming . . . 4
5.3 Float-only evaluation . . . 5
5.4 Training pipeline . . . 5

6 Experimental design **6**

7 Results **7**
7.1 The two axes are complementary, not additive . . . 8
7.2 A lighter joint configuration substantially reduces the accuracy cost . . . 8
7.3 Recommendation . . . 9

8 Discussion and limitations **9**

9 Conclusion **9**

1 Introduction

Brainchip's Akida 1 neuromorphic processor accelerates convolutional neural network inference by skipping computation on zero-valued activations as data flows through the network event by event. Because this speedup comes specifically from *activation* sparsity, work on making VWW models more efficient on Akida 1 has so far focused on that single axis: adding an activity regularizer to every ReLU layer during training so that more of its outputs are driven to exactly zero.

A separate, and largely independent, lever is *structured sparsity*: pruning whole output channels (filters) from convolutional layers. Unlike activation sparsity, channel pruning does not by itself make Akida 1 run any faster — the hardware does not accelerate on weight sparsity — but it directly reduces the number of parameters the model needs to store, which matters for memory footprint on constrained edge devices.

This report documents an experiment that trains VWW with both regularization mechanisms active at once, to answer two questions:

(i) Does combining activation-sparsity regularization with structured channel pruning cost *more* accuracy than the sum of what each costs on its own, or can the two be pushed together more cheaply?

(ii) Starting from an aggressive joint configuration, can a better-optimized operating point be found that keeps most of the sparsity/size benefit while shrinking the accuracy drop?

The short answers, developed in Section 7, are: the two axes are complementary (question i), and yes — a lighter joint configuration cuts the accuracy drop by more than 3× while retaining most of the benefit (question ii).

All work in this report is on branch `akida1-sparsify-vww-joint`, a follow-up to the single-axis activation-sparsity experiment on branch `akida1-sparsify-vww` (`SPARSITY_EXPERIMENT.md`).

2 Background

2.1 Akida 1 and activation sparsity

Akida 1 is an event-based neural processor: rather than computing every multiply-accumulate in a dense feature map regardless of value, it skips work corresponding to zero activations. A model with sparser intermediate activations therefore runs measurably faster and at lower power on the physical device. This motivates training procedures that explicitly encourage activations — specifically, the outputs of ReLU layers — toward zero, provided this can be done without an unacceptable loss of accuracy.

Weight (parameter) sparsity is a different property: it describes how many of the trained weights themselves are zero (or, in the case of structured/channel pruning, how many whole channels have been removed). Akida 1's execution model does not exploit this for speed. Reducing the parameter count is nonetheless useful on its own terms, primarily for reducing the model's memory footprint.

2.2 Visual Wake Words

Visual Wake Words (VWW) is a binary image classification task: given a 96×96 RGB image, predict whether a person is present (`person`) or not (`non_person`). It is a standard microcontroller-class vision benchmark intended to be representative of always-on, resource-constrained vision workloads.

3 Dataset

The dataset used is `vw_coco2014_96`, a 96×96 pre-cropped/resized derivative of the COCO 2014 dataset labeled for the person / non-person VWW task, obtained from Silicon Labs' public machine-learning benchmark mirror. On disk it is organized as two class subdirectories (`person/`, `non_person/`).

Table 1: Dataset summary, as loaded by `vww_data.get_data`.

Property	Value
Classes	2 (`person`, `non_person`)
Input resolution	$96 \times 96 \times 3$ (RGB, `uint8`)
Train / validation split	90% / 10%
Training images	98,658
Validation images	10,961
Train-time augmentation	rotation ($\pm 10°$), width/height shift (5%), zoom (10%), horizontal flip
Validation-time augmentation	none (only the train/val split is applied)

Loading is performed with Keras' `ImageDataGenerator.flow_from_directory`, using a fixed random seed (42) so the train/validation split and augmentation stream are reproducible across runs.

4 Model architecture

The model is AkidaNet at width multiplier $\alpha = 0.25$, a MobileNetV1-style convolutional network optimized for Akida 1, initialized from ImageNet-pretrained weights (`akidanet_imagenet_pretrained` with `alpha=0.25`) and adapted for VWW by taking the penultimate feature layer and attaching a 2-class classification head (`vww_model.py`). The network is a strict feed-forward chain with no residual/skip connections: every layer takes only its immediate predecessor's output as input, which is what makes the channel-pruning procedure in Section 5.2 tractable.

Every convolutional block follows the same pattern: a (separable) convolution, a `BatchNormalization` layer, and a clipped ReLU activation (`ReLU3.75`). The full layer sequence, with output shapes and parameter counts as reported by `model.summary()`, is given in Table 2.

Table 2: AkidaNet-VWW ($\alpha = 0.25$) architecture. Every `conv_*`/`separable_*` row is immediately followed by its own BatchNormalization and ReLU (omitted from the table for brevity; their parameter counts are included in the running total).

Layer	Type	Output shape	Params (layer + BN)
`input`	Input	$96 \times 96 \times 3$	0
`rescaling`	Rescaling ($\div 255$)	$96 \times 96 \times 3$	0
`conv_0`	Conv2D, stride 2	$48 \times 48 \times 8$	248
`conv_1`	Conv2D	$48 \times 48 \times 16$	1,216
`conv_2`	Conv2D, stride 2	$24 \times 24 \times 32$	4,736
`conv_3`	Conv2D	$24 \times 24 \times 32$	9,344
`separable_4`	SeparableConv2D, stride 2	$12 \times 12 \times 64$	2,592
`separable_5`	SeparableConv2D	$12 \times 12 \times 64$	4,928
`separable_6`	SeparableConv2D, stride 2	$6 \times 6 \times 128$	9,280
`separable_7`	SeparableConv2D	$6 \times 6 \times 128$	18,048
`separable_8`	SeparableConv2D	$6 \times 6 \times 128$	18,048
`separable_9`	SeparableConv2D	$6 \times 6 \times 128$	18,048
`separable_10`	SeparableConv2D	$6 \times 6 \times 128$	18,048
`separable_11`	SeparableConv2D	$6 \times 6 \times 128$	18,048
`separable_12`	SeparableConv2D, stride 2	$3 \times 3 \times 256$	34,944
`separable_13`	SeparableConv2D + global-avg-pool	256	68,864
`predictions`	Dense (logits, no activation)	2	514
		Total	**226,906**
		Trainable / non-trainable	223,914 / 2,992

`separable_13` additionally performs a global average pool before its BatchNormalization, collapsing the 3×3 spatial map to a 256-vector that is what the classification head (`predictions`) reads directly.

5 Methodology

Two sparsity mechanisms are combined: an established activation-sparsity regularizer (Section 5.1) and a newly implemented structured-sparsity (channel pruning) procedure (Section 5.2).

5.1 Activation sparsity: normalized Hoyer-Square regularization

Following the prior single-axis experiment, activation sparsity is induced with a Hoyer-Square activity regularizer [2] attached to every ReLU layer's output x:

$$\mathcal{L}_{\text{Hoyer}}(x) = \lambda \cdot \frac{(\sum_i |x_i|)^2}{\sum_i x_i^2 + \varepsilon}, \tag{1}$$

scale-invariant and sparsity-inducing: for a fixed L_1 norm, a configuration with a few large activations and the rest at zero has a lower Hoyer-Square value than one with many small, non-zero activations. The *normalized* variant used throughout this report additionally divides by the tensor's element count N,

$$\mathcal{L}_{\text{Hoyer-norm}}(x) = \frac{\lambda}{N} \cdot \frac{(\sum_i |x_i|)^2}{\sum_i x_i^2 + \varepsilon}, \tag{2}$$

which bounds the penalty to $(0, 1]$ regardless of layer width and avoids applying disproportionate pressure to the network's largest layers (see `SPARSITY_EXPERIMENT.md` for the calibration study motivating this). The prior experiment established $\lambda = 3.6$ as the best point on this axis alone: 70.4% activation sparsity (as measured on the quantized, Akida-converted model) for a 2.6-point accuracy cost.

5.2 Structured sparsity: Network Slimming

No channel-pruning tooling exists anywhere in `akida_models`, `quantizeml`, or `cnn2snn` (confirmed by inspecting the installed packages), so this was implemented from scratch, following the Network Slimming method of Liu et al. [1].

Channel-importance signal. During training, an L_1 penalty is added on the γ (scale) parameter of each targeted `BatchNormalization` layer:

$$\mathcal{L}_\gamma = \mu \sum_c |\gamma_c|, \tag{3}$$

summed over the channels c of one BN layer, with $\mu = 10^{-5}$ held fixed throughout this report. Because `BatchNormalization` layers built by `akida_models.layer_blocks` are not given a `gamma_regularizer` at construction time, this penalty is instead attached directly as a model-level loss via the zero-argument-callable form of `model.add_loss`, evaluated fresh at every training step against the live `gamma` variable.

Pruning. After training, channels of each targeted `SeparableConv2D` layer are ranked by the magnitude of their paired BN layer's γ, and the lowest-ranked fraction (the *prune target*) is dropped uniformly per layer. The model is then physically rebuilt smaller: the pruned layer's pointwise kernel and bias are sliced along the output-channel axis, its BN's four statistics are sliced identically, and — since the architecture is a strict chain with no skip connections (Section 4) — the very next weighted layer has its depthwise and pointwise kernels sliced along the corresponding *input*-channel axis to match. Every other layer (ReLU, pooling, input scaling, the classification head) is rebuilt unchanged once no pending channel change remains to propagate.

Pruning scope. Only `separable_4` through `separable_12` (9 of the network's 10 separable-convolution blocks) are eligible for pruning. The stem (`conv_0`–`conv_3`) and `separable_13` are excluded: the classification head reads `separable_13/relu` directly, so pruning it would additionally require resizing the `predictions` Dense layer's input — extra engineering risk not needed to answer the questions this report addresses.

5.3 Float-only evaluation

This study is conducted entirely on float (unquantized, unconverted) models — no `cnn2snn quantize`, quantization-aware fine-tuning, or `convert` to an Akida model is performed anywhere in the pipeline. Two considerations motivate this:

(a) The prior single-axis experiment found that Akida's software behavioral simulator and its hardware execution engine are not bit-exact at the activation-threshold level, producing an internally consistent but not directly hardware-comparable sparsity number when no physical device is attached. Measuring sparsity directly on the float model's ReLU outputs sidesteps this discrepancy entirely.

(b) Restricting the pipeline to float training keeps the channel-pruning implementation decoupled from any quantization/hardware-mapping constraints, which was out of scope for this study.

Activation sparsity is accordingly computed as the mean, across all 14 ReLU layers, of the fraction of exactly-zero output elements, evaluated on 1,000 held-out samples. **This number is not directly comparable to the prior experiment's Akida-measured sparsity** — at the same regularizer strength ($\lambda = 3.6$), the float measurement reads 80.1% versus 70.4% on the quantized/converted model — but is internally consistent for comparing configurations within this report.

5.4 Training pipeline

Each configuration is defined by a pair (λ, p): the activation-regularizer strength and the structured prune target. Training proceeds in up to two phases:

Phase 1 (float training, 40 epochs). The pretrained model is built; if $\lambda > 0$, the Hoyer-Square-normalized regularizer is attached to every ReLU layer's `activity_regularizer`; if $p > 0$, the BN-γ L_1 loss is attached to every prunable layer. The model is trained with Adam and a cosine-decay learning-rate schedule with a 10%-of-training linear warmup, peak learning rate 10^{-3}.

Pruning (if $p > 0$). Channels are pruned per Section 5.2 and the model rebuilt.

Phase 2 (post-prune fine-tuning, 15 epochs, only if $p > 0$). The rebuilt model is fine-tuned at a lower learning rate (10^{-4}) to recover accuracy lost to channel removal, with the activation regularizer re-attached (rebuilding creates fresh layer instances that do not carry it over automatically).

Batch size is 32 throughout; all runs use random seed 42 for reproducibility.

Data-loading performance. An early timing measurement found that a single training epoch took approximately 30 minutes on a GPU, with the GPU itself largely idle — the bottleneck was Keras' single-threaded `ImageDataGenerator`, which could not perform image decoding and augmentation fast enough to keep the accelerator fed. Enabling parallel data loading (`workers=8`, `use_multiprocessing=True`, `max_queue_size=32` on `model.fit`) reduced this to approximately 4 minutes per epoch, a 7.5× speedup, which is what made the sweep described below practical within the time available.

6 Experimental design

Two rounds of configurations were run, sharing the same summary CSV so later points build on earlier ones without re-running them.

Pilot (2×2 grid). A first, deliberately small grid tests all four combinations of "activation regularizer on/off" × "pruning on/off", at the strongest settings considered: $\lambda = 3.6$ (the established best point from the single-axis experiment) and $p = 40\%$ (a starting guess, with no prior data point on this axis).

Follow-up (lighter settings). Motivated by the pilot's finding that the joint configuration's combined accuracy cost, while sub-additive, was still a 3.90-point drop, four additional points probe lighter regularization: $\lambda = 1.5$ and $p = 20\%$ individually and jointly, plus one intermediate joint point at $\lambda = 2.5$, $p = 30\%$, to trace the shape of the accuracy/sparsity/size trade-off between the light and aggressive joint settings.

Table 3 summarizes all eight configurations.

Table 3: All configurations run. λ is the normalized Hoyer-Square strength; p is the structured prune target (fraction of channels dropped per targeted layer).

Tag	λ	p
`baseline`	0	0%
`activation_only`	3.6	0%
`structured_only`	0	40%
`joint`	3.6	40%
`activation_light`	1.5	0%
`structured_light`	0	20%
`joint_light`	1.5	20%
`joint_medium`	2.5	30%

All runs used GPU 1 of a shared training server (`NVIDIA A30`); the pilot took approximately 1 hour 40 minutes end to end, the follow-up a further 2 hours 10 minutes.

7 Results

Table 4: Full results. Accuracy drop is relative to `baseline` (89.14%). Parameter reduction is relative to `baseline`'s 226,906 parameters. Activation sparsity is the float-model measurement described in Section 4.3 and is *not* directly comparable to the prior experiment's Akida-measured numbers.

Tag	λ	p	Accuracy	Acc. drop	Act. sparsity	Params (reduction)
`baseline`	0	0%	89.14%	–	51.9%	226,906 (–)
`activation_only`	3.6	0%	87.32%	1.82pt	80.1%	226,906 (0%)
`activation_light`	1.5	0%	88.17%	0.98pt	71.1%	226,906 (0%)
`structured_only`	0	40%	86.11%	3.04pt	54.0%	112,879 (50.3%)
`structured_light`	0	20%	88.45%	0.69pt	52.9%	164,162 (27.7%)
`joint`	3.6	40%	85.25%	3.90pt	77.6%	112,879 (50.3%)
`joint_medium`	2.5	30%	87.06%	2.08pt	74.7%	137,836 (39.3%)
`joint_light`	1.5	20%	87.92%	1.22pt	69.4%	164,162 (27.7%)

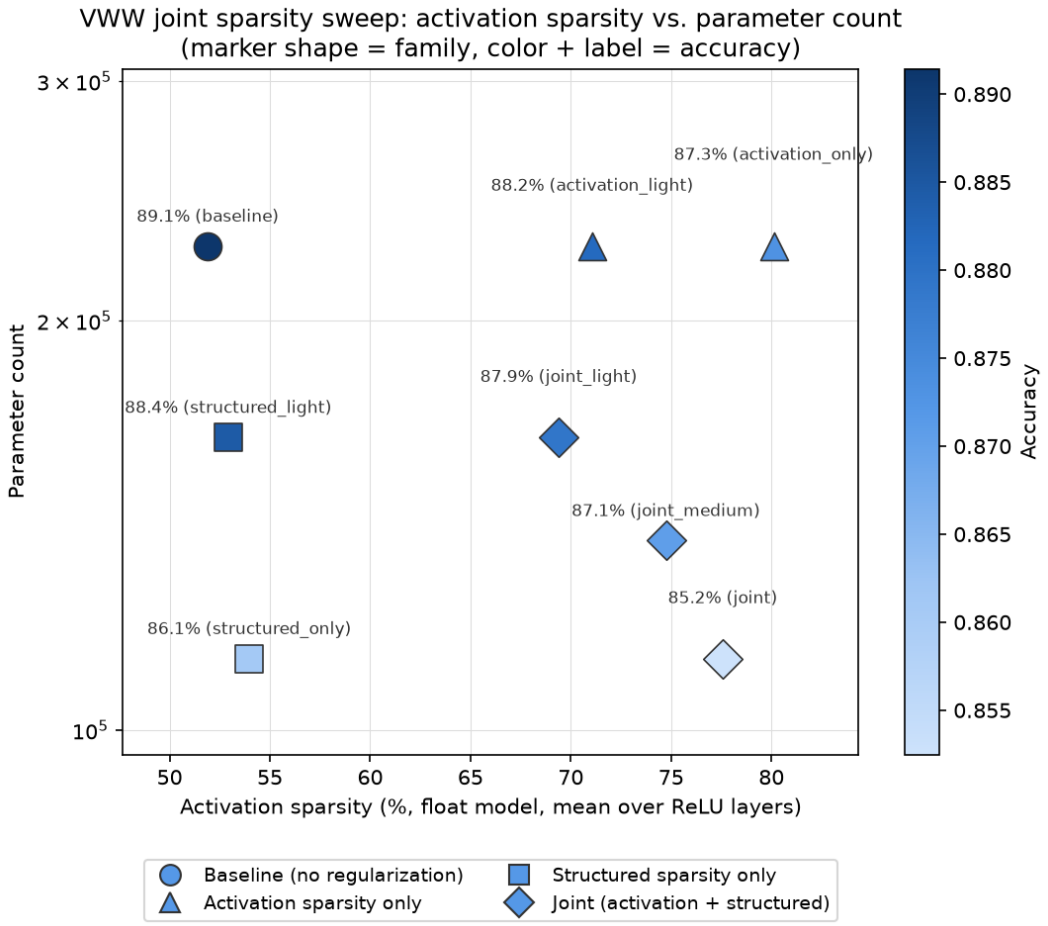

Figure 1: Activation sparsity vs. parameter count for all eight configurations. Marker shape encodes family (circle: baseline, triangle: activation sparsity only, square: structured sparsity only, diamond: joint); marker color and the accompanying label encode accuracy.

7.1 The two axes are complementary, not additive

Relative to `baseline`, `activation_only` alone costs 1.82 accuracy points and `structured_only` alone costs 3.04 points; a naive expectation that the costs simply add would predict roughly $1.82 + 3.04 = 4.86$ points for doing both. The aggressive `joint` configuration instead costs 3.90 points — less than the sum of its parts — while still reaching 77.6% activation sparsity (close to `activation_only`'s own 80.1%) and the full 50.3% parameter reduction that `structured_only` achieves on its own (parameter count is set entirely by the prune target, independent of the activation regularizer). Pushing both sparsity mechanisms at once is therefore a good trade, not a compounding cost.

7.2 A lighter joint configuration substantially reduces the accuracy cost

Halving both regularization strengths ($\lambda : 3.6 \to 1.5$, $p : 40\% \to 20\%$) reduces the joint configuration's accuracy cost from 3.90 points to just **1.22 points** (`joint_light`), while retaining 69.4% activation sparsity and a 27.7% parameter reduction. The three joint operating points obtained so far — `joint_light` (1.22pt / 69.4% / −27.7%), `joint_medium` (2.08pt / 74.7% / −39.3%), and

`joint` (3.90pt / 77.6% / −50.3%) — trace a smooth, gradually-degrading curve rather than a sudden cliff, consistent with the shape each regularizer shows individually.

7.3 Recommendation

`joint_light` ($\lambda = 1.5$, $p = 20\%$) is the best operating point found in this study when minimizing accuracy loss is the priority: 87.92% accuracy (a 1.22-point drop from baseline) with a meaningfully smaller (27.7% fewer parameters) and meaningfully sparser (69.4% vs. 51.9% baseline) model. The heavier `joint_medium` and `joint` configurations remain preferable only if the priority shifts toward maximizing sparsity or size reduction and a larger accuracy budget is acceptable.

8 Discussion and limitations

Float-vs-hardware sparsity gap. At matched regularizer strength ($\lambda = 3.6$), the float-model activation-sparsity measurement used throughout this report (80.1%) is substantially higher than the prior experiment's Akida-hardware/software-simulator-measured figure for the same regularizer (70.4%). This is expected — the two measurements are taken on different representations of the model (unquantized float vs. quantized/converted) using different execution engines — and means the absolute sparsity percentages in this report should be read as internally comparable to each other, not to the prior experiment's numbers.

No hardware latency/power validation. Because this study is float-only, it makes no claim about actual Akida 1 inference latency or power. Akida 1 does not accelerate on weight sparsity in the first place, so the structured-sparsity axis's benefit here is entirely a model-footprint (parameter/memory) argument, not a runtime one. Validating the joint approach's practical benefit end-to-end would require carrying a chosen configuration through quantization, quantization-aware fine-tuning, and conversion to an Akida model, then benchmarking on physical hardware, as the original single-axis experiment did for its own best point.

Unexplored surface. The three joint configurations tested move both knobs together in fixed proportion; no configuration with, say, high λ paired with low p (or vice versa) has been tried, so it is not yet known whether the two axes trade off symmetrically once combined. The BN-γ regularization strength $\mu = 10^{-5}$ was likewise held fixed throughout and has not itself been tuned. Finally, nothing lighter than `joint_light` has been tested; if an even smaller accuracy drop is desired, the natural next step is to continue in the same direction (e.g. $\lambda \approx 0.5$–1, $p \approx 10$–15%) rather than explore a new direction.

9 Conclusion

Activation sparsity and structured (channel) sparsity are two largely independent levers for making the VWW AkidaNet model more efficient: the former reduces Akida 1 inference time via event-driven compute skipping, the latter reduces the model's parameter footprint. This study shows that the two can be pushed simultaneously at a combined accuracy cost lower than their sum, and that a moderate, jointly-tuned configuration (`joint_light`: $\lambda = 1.5$ activation regularization, 20% structured channel pruning) recovers most of both benefits — 69.4% activation sparsity and a 27.7% parameter reduction — for only a 1.22-point accuracy drop from the unregularized baseline, making

it the recommended starting point for further work, including full quantization and on-hardware validation.

References

[1] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang. Learning Efficient Convolutional Networks through Network Slimming. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

[2] H. Yang, W. Wen, and H. Li. DeepHoyer: Learning Sparser Neural Network with Differentiable Scale-Invariant Sparsity Measures. In *International Conference on Learning Representations (ICLR)*, 2020.

[3] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv preprint arXiv:1704.04861*, 2017.

[4] A. Chowdhery, P. Warden, J. Shlens, A. Howard, and R. Rhodes. Visual Wake Words Dataset. *arXiv preprint arXiv:1906.05721*, 2019.